

UNIVERSITY of CALIFORNIA
SANTA CRUZ

**A STUDY OF PHYSICS-INFORMED MACHINE LEARNING
THROUGH THE ONE-DIMENSIONAL HEAT EQUATION**

A thesis submitted in partial satisfaction of the
requirements for the degree of

BACHELOR OF SCIENCE

in

COMPUTATIONAL PHYSICS

by

James Desjarlais

March 2026

The thesis of James Desjarlais is approved by:

Professor Stefano Profumo
Advisor

Professor Onuttom Narayan
Chair, Department of Physics

Copyright © by

James Desjarlais

2026

Abstract

A Study of Physics-Informed Machine Learning through the One-Dimensional Heat Equation

by

James Desjarlais

Physics-informed neural networks (PINNs) have emerged as a promising mesh-free framework for solving partial differential equations by embedding governing physical laws, in the form of a partial differential equation, directly into the training process. While PINNs offer flexibility in handling high-dimensional systems, complex geometries, and inverse problems, their practical reliability and performance relative to classical numerical methods remain active areas of research. In this paper, we examine the strengths and limitations of PINNs through a focused review and computational investigation through the one-dimensional heat equation. We implement a standard PINN to solve the forward heat-diffusion problem and compare its behavior, accuracy, and computational cost to both the analytical solution and a classical finite-difference baseline. We also introduce the inverse-problem formulation to illustrate the capacity of the same framework to be extended to infer unknown physical parameters from limited data. Together, these results provide a practical benchmark for understanding the tradeoffs between PINNs and classical PDE solvers.

Contents

Figures	vi
Tables	vii
1 Introduction	viii
1.1 What PINNs are	viii
1.2 Why PINNs are interesting	ix
1.3 Comparison to standard numerical solvers	ix
1.4 Contributions	x
2 Background	xi
2.1 1-D heat equation and analytical reference	xi
2.2 Classical numerical baseline	xii
2.3 PINNs: Core Mechanism	xiii
2.4 PINNs: Applications and Limitations	xv
3 Experimental Setup	xvii
3.1 Problem definition and constraints	xvii
3.2 What gets measured	xvii
3.3 Fair comparison rules	xviii
4 Finite-difference Baseline	xix
5 PINN Implementation	xx
5.1 Network Architecture	xx
5.2 Loss Construction	xxi
5.3 Sampling Strategy	xxii
5.4 Optimization Details	xxii

6	Forward Problem Results	xxiv
6.1	Qualitative solution comparison	xxv
6.2	Quantitative Accuracy and Runtime Comparison	xxvi
7	Inverse Problem	xxix
7.1	Inverse problem: formulation and setup	xxix
7.2	Loss function for the inverse problem	xxx
7.3	Discussion and future work	xxxi
8	Conclusion	xxxii
9	Bibliography	xxxiv

Figures

5.1	Training curves illustrating convergence to a minimum in the loss function through Adam and L-BGFS	xxiii
6.1	Heat map plots of the exact solution in comparison to the PINN and FD solutions.	xxv
6.2	Absolute error relative to the analytical solution for the PINN and Crank–Nicolson (FD) methods over the spacetime domain.	xxvi

Tables

4.1	Baseline solver accuracy relative to the analytical solution.	xix
6.1	Summary of experimental settings.	xxiv
6.2	Compute cost and error comparison for the forward problem.	xxviii

1 Introduction

1.1 What PINNs are

Physics-informed neural networks (PINNs) are a class of machine learning models designed to solve differential equations by embedding known mathematical or physical constraints in the form of a partial differential equation (PDE) directly into the training process. Rather than relying on large labeled datasets of input-output pairs as in traditional supervised learning, PINNs minimize a composite loss function that penalizes violations of the governing PDE, as well as specified initial and boundary conditions. By enforcing the structure of the underlying equation, PINNs are able to learn meaningful solutions with little or no external data. Derivatives appearing in the PDE are computed internally using a process called automatic differentiation (autodiff) at points sampled throughout the domain. This allows differences from the right and left side of the equation to be evaluated and minimized during training. Despite the name, this framework is broadly applicable to problems governed by differential equations beyond just the field of Physics [1, 2].

1.2 Why PINNs are interesting

In recent years, PINNs have attracted significant attention as a mesh-free alternative to traditional numerical solvers. Their ability to handle high-dimensional systems, sparse data, and inverse problems has made them particularly appealing in settings where classical solvers become computationally expensive or difficult to implement [1, 2, 3].

1.3 Comparison to standard numerical solvers

Despite their conceptual promise, PINNs also present practical challenges. Training can be sensitive to loss weighting, sampling strategy, and the use of high-order derivative terms, which can lead to unstable or inconsistent convergence. As a result, understanding the benefits and limits of PINNs remains an active area of research [4, 2].

The well known heat equation provides an ideal framework for studying the behavior of PINNs. As a linear parabolic PDE with known analytical solutions under standard boundary and initial conditions, it allows for a direct comparison between PINN predictions and analytical solutions. At the same time, the equation captures many of the core elements present in more complex physical systems, including diffusion, time evolution, and parameter dependence. This makes it a useful example for isolating the strengths and limitations of the PINN framework without introducing unnecessary mathematical complexity.

1.4 Contributions

This paper presents a review of physics-informed neural networks together with a practical implementation for the one-dimensional heat equation. A PINN solution to the forward problem is developed and compared with a classical finite-difference baseline to assess qualitative behavior, quantitative accuracy, and computational runtime. The inverse-problem is also introduced to demonstrate theoretical capacity of how the PINN framework can be extended to estimate unknown physical parameters. Overall, this paper highlights the practical tradeoffs between PINNs and classical PDE solvers.

2 Background

2.1 1-D heat equation and analytical reference

We consider the one-dimensional heat equation on a rod of unit-length.

$$\frac{\partial u}{\partial t}(x, t) = \alpha \frac{\partial^2 u}{\partial x^2}(x, t), \quad x \in [0, 1], \quad t \in [0, T], \quad (2.1)$$

with Dirichlet boundary conditions

$$u(0, t) = 0, \quad u(1, t) = 0, \quad (2.2)$$

and initial condition

$$u(x, 0) = \sin(\pi x). \quad (2.3)$$

For this particular setup, the solution is as follows:

$$u_{\text{true}}(x, t) = e^{-\alpha\pi^2 t} \sin(\pi x), \quad (2.4)$$

which follows from separation of variables and satisfies the Dirichlet boundary conditions. We use (2.4) as a ground-truth reference for error evaluation.

To quantify accuracy, we evaluate predictions on a fixed spacetime grid $\{(x_i, t_j)\}$ and report relative L^2 and L^∞ error metrics where

$$\text{Rel } L_2 = \frac{\|e\|_2}{\|u_{\text{true}}\|_2} = \frac{\left(\sum_{i=1}^{N_x} \sum_{j=1}^{N_t} (u_{\text{pred}}(x_i, t_j) - u_{\text{true}}(x_i, t_j))^2 \right)^{1/2}}{\left(\sum_{i=1}^{N_x} \sum_{j=1}^{N_t} (u_{\text{true}}(x_i, t_j))^2 \right)^{1/2}} \quad (2.5)$$

and

$$L_\infty = \max_{i,j} |u_{\text{pred}}(x_i, t_j) - u_{\text{true}}(x_i, t_j)|. \quad (2.6)$$

2.2 Classical numerical baseline

We also solve the heat equation with a standard finite-difference method as a baseline for comparison. We discretize the domain with

$$x_i = i\Delta x \quad (i = 0, \dots, N_x), \quad t_n = n\Delta t \quad (n = 0, \dots, N_t),$$

and denote $u_{i,n} \approx u(x_i, t_n)$.

A simple explicit method is Forward-Time Centered-Space (FTCS),

$$u_{i,n+1} = u_{i,n} + r(u_{i+1,n} - 2u_{i,n} + u_{i-1,n}), \quad r = \alpha \frac{\Delta t}{\Delta x^2}. \quad (2.7)$$

FTCS is easy to implement but only stable when $r \leq \frac{1}{2}$, which can force very small Δt on fine grids.

To avoid this limitation, we implement a stronger baseline known as the Crank–Nicolson (CN) method. Rather than evaluating the spatial second derivative entirely at the current time step, CN averages this term between the current and next time step. This averaging improves the overall accuracy of the method and leads to second-order accuracy in both space and time. In matrix form, the method may be written as,

$$\left(I - \frac{r}{2}A\right) \vec{\mathbf{u}}_{n+1} = \left(I + \frac{r}{2}A\right) \vec{\mathbf{u}}_n, \quad (2.8)$$

where $\vec{\mathbf{u}}_n = (u_{1,n}, \dots, u_{N_x-1,n})$ is the temperature vector at a given time step n , I is the identity matrix and A is the standard second-difference matrix. We use this solver as the classical reference when comparing accuracy and runtime against the PINN.

2.3 PINNs: Core Mechanism

A physics-informed neural network (PINN) works by giving a prediction $u_\theta(x, t)$ and training it to approximate the exact solution $u(x, t)$. In typical supervised learning with standard labeled input-output pairs, a network learns by minimizing the error in its guesses for the output. However, with a PINN, we set up a specialized loss function that punishes deviations from the constraints of the governing PDE. In this particular setup, our loss function is made up of constraints from the PDE, the

initial condition, and the boundary conditions [1, 2].

For the PDE, we have a loss term based off of the residual of the PDE. Simply put, the residual is just one side of an equation subtracted from the other. Since we know that for a perfect solution to a given equation the residual should equal zero, we train to drive this number as close to zero as possible. Similarly, we fit $u_\theta(x, 0)$ to the given initial condition and $u_\theta(0, t) = u_\theta(1, t)$ to the given boundary condition(s). In practice, this is implemented as a weighted sum of mean-squared errors [1, 2].

The derivatives in the PDE are computed through a process called automatic differentiation (autodiff). During the forward pass, autodiff records the operations used to build the computational graph. A backward pass is then performed, repeatedly applying the chain rule to compute the required derivatives [5].

In practice, the network starts by making an initial guess for $u_\theta(x, t)$. This guess is formed by passing the inputs through the multiple layers of the neural network. In the input layer, the network takes in the variables x and t and produces a vector $a^{(1)}$, whose length is equal to the number of nodes in the first hidden layer, where

$$a_j^{(1)} = \tanh \left(w_{0,j}^{(1)} + w_{1,j}^{(1)}x + w_{2,j}^{(1)}t \right), \quad j = 1, \dots, 20, \quad (2.9)$$

given the use of tanh activation functions.

From there, the network passes this vector through the hidden layers, building progressively more complex vector representations. For each succeeding layer,

$$a_j^{(l)} = \tanh \left(w_{0,j}^{(l)} + \sum_{i=1}^{20} w_{i,j}^{(l)} a_i^{(l-1)} \right), \quad j = 1, \dots, 20, \quad l = 2, 3, 4. \quad (2.10)$$

Finally, the network reaches the output layer, where it produces its guess approximation for $u_\theta(x, t)$ by taking a weighted sum of the activations from the final hidden layer, where

$$u_\theta(x, t) = w_0^{(5)} + \sum_{i=1}^{20} w_i^{(5)} a_i^{(4)}. \quad (2.11)$$

Through training, the network optimizes all of the weights $w_{i,j}^{(l)}$ and $w_{0,j}^{(l)}$ in accordance with the loss function to finally construct a sufficiently accurate approximation of the analytical solution.

2.4 PINNs: Applications and Limitations

One appealing feature of PINNs is flexibility: once trained, u_θ gives a continuous approximation that can be evaluated at any (x, t) without time-stepping as opposed to traditional meshed solvers. Since the governing equation is enforced during training, PINNs also extend naturally to inverse problems by treating unknown physical parameters (like α) as trainable variables [1, 2].

That said, PINNs suffer from many major pitfalls. With a loss function that has multiple terms focused on different aspects of the system certain terms can overwhelm

the other. For example, if the PDE term dominates, the network may ignore boundary/initial conditions or if the constraint terms dominate, it may fit the boundaries but not the PDE well. For training, the position and number of collocation points change results a lot, especially near regions where the solution changes quickly [4].

Because of these trade-offs, it is useful to benchmark PINNs against a classical solver on a problem with a known analytical reference, where accuracy and compute cost can be measured cleanly.

3 Experimental Setup

3.1 Problem definition and constraints

We study the one-dimensional heat equation on $x \in [0, 1]$ and $t \in [0, T]$ with homogeneous Dirichlet boundary conditions and initial condition $u(x, 0) = \sin(\pi x)$. For the duration of this paper, we set $\alpha = 0.4$ and $T = 1$. As discussed in Chapter 2, this setup has a straightforward closed-form solution $u_{\text{true}}(x, t) = e^{-\alpha\pi^2 t} \sin(\pi x)$.

To compare methods consistently, we evaluate each method on a fixed spacetime grid $\{(x_i, t_j)\}$ that is independent of the PINN collocation points. All experiments were run using PyTorch. Automatic differentiation is used to compute derivatives for the PINN [5].

3.2 What gets measured

Accuracy is reported using the relative L_2 error and the maximum error L_∞ defined in Chapter 2, computed over the evaluation grid. Compute cost is measured

through Python’s time library. We also record training curves (total loss and individual components) to illustrate stability and convergence, since PINNs are known to be sensitive to training choices [4, 2].

3.3 Fair comparison rules

Both methods solve the same PDE with the same α , domain, and IC/BC. Errors are always computed on the same fixed grid. These specifications ensure our comparisons between computational methods remain fair and interpretable.

4 Finite-difference Baseline

As introduced in Section 2.2, the Crank–Nicolson method is used in this work as the classical numerical baseline for the forward heat equation. Its purpose here is not to introduce a new finite-difference method, but to provide a reliable reference for later comparison with the PINN in both accuracy and runtime.

To verify that the implementation behaves as expected, the Crank–Nicolson solution is compared against the analytical solution for the same problem setup used throughout this study. The resulting error metrics, reported in Table 4.1, show only very small discrepancies. This confirms that the finite-difference solver serves as an appropriate baseline for the forward-problem comparisons presented in Chapter 6 [6, 7].

Table 4.1: Baseline solver accuracy relative to the analytical solution.

Method	Relative L^2 error	L^∞ error
Finite difference (CN)	3.3437×10^{-7}	1.7558×10^{-7}

5 PINN Implementation

5.1 Network Architecture

We approximate the solution using a neural network $u_\theta(x, t)$ that takes (x, t) as input and outputs a scalar temperature prediction u . The model is a fully-connected multilayer perceptron with four hidden layers of width 20 and tanh activations. Concretely, the architecture is

$$(x, t) \in R^2 \longrightarrow 20 \longrightarrow 20 \longrightarrow 20 \longrightarrow 20 \longrightarrow u \in R,$$

with $\tanh()$ applied after each hidden linear layer and a linear output layer. This choice follows the standard baseline PINN setup used in many introductory implementations [1, 2].

5.2 Loss Construction

Following the PINN framework, we define the heat-equation residual

$$res_{\theta}(x, t) = \frac{\partial u_{\theta}}{\partial t}(x, t) - \alpha \frac{\partial^2 u_{\theta}}{\partial x^2}(x, t), \quad (5.1)$$

where derivatives are computed using automatic differentiation [5]. Training minimizes a composite mean-squared-error loss with three components [1, 2]:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{k=1}^{N_f} (res_{\theta}(x_k, t_k))^2, \quad (5.2)$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_{bc}} \sum_{k=1}^{N_{bc}} (u_{\theta}(x_k, t_k))^2 \quad \text{evaluated at } x = 0, 1 \quad (5.3)$$

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{ic}} \sum_{k=1}^{N_{ic}} (u_{\theta}(x_k, 0) - \sin(\pi x_k))^2. \quad (5.4)$$

For the duration of experiments we use equal weighting (λ). As a result we get our composite loss function as a weighted sum of these terms:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{PDE}} + \lambda_2 \mathcal{L}_{\text{BC}} + \lambda_3 \mathcal{L}_{\text{IC}} \quad (5.5)$$

where

$$\lambda_1 = \lambda_2 = \lambda_3 = 1. \quad (5.6)$$

5.3 Sampling Strategy

Training points are sampled once at the start of training and held fixed. Interior collocation points (x_f, t_f) are sampled uniformly from $(0, 1) \times (0, T)$ with $N_f = 10,000$. Boundary points are sampled on $x = 0$ and $x = 1$ with $N_{bc} = 200$ points on each boundary (400 total), with times sampled uniformly from $[0, T]$. Initial-condition points are sampled with $N_{ic} = 200$ points uniformly in $x \in [0, 1]$ at $t = 0$ [4, 8].

5.4 Optimization Details

We train using a two-stage optimization strategy that is common in PINN implementations [1, 2]. First, we optimize with Adam (learning rate 10^{-3}) for 2000 iterations. We then refine the solution with L-BFGS for up to 500 iterations. The motivation for this choice is that Adam is effective in the initial stage of training by using gradient information to update the weights in the direction of steepest local decrease, L-BFGS is implemented later because it uses approximate curvature information to navigate flatter regions of the loss landscape more effectively.

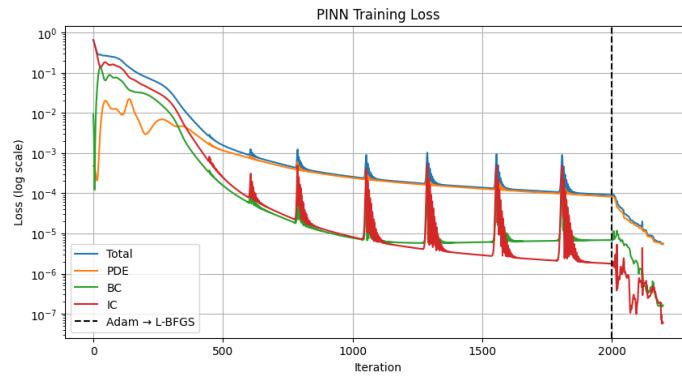


Figure 5.1: Training curves illustrating convergence to a minimum in the loss function through Adam and L-BGFS

6 Forward Problem Results

This chapter compares the forward-problem results for the physics-informed neural network (PINN) with the Crank-Nicolson (CN) finite-difference baseline for the one-dimensional heat equation. Table 6.1 summarizes the numerical settings used in this comparison. The discussion begins with a qualitative comparison of the computed and analytical solutions, followed by quantitative accuracy and runtime results.

Table 6.1: Summary of experimental settings.

Item	Value
Domain	$x \in [0, 1], t \in [0, T]$
α	0.4
Evaluation grid	$N_x = 1000, N_t = 1000$
Finite difference scheme	Crank-Nicolson (CN)
PINN architecture	4 hidden layers / width 20 / tanh activation
Optimizer	Adam (lr = 1e-3) + L-BFGS
Collocation points	$N_{pde} = 10,000, N_{bc} = 400, N_{ic} = 200$
Loss weights	$\lambda_1 = \lambda_2 = \lambda_3 = 1$
Hardware	CPU

6.1 Qualitative solution comparison

We first compare the analytical solution $u_{\text{true}}(x, t)$ to the learned PINN solution $u_{\theta}(x, t)$ and the finite-difference solution $u_{fd}(x, t)$. Figure 6.1 shows a heatmap of the analytical solution as well as the approximate temperature field from both computational methods. Visually, the three plots are practically indistinguishable. Both methods seemingly capture the expected diffusion behavior, i.e. the initial sine profile smoothly decays over time while maintaining the boundary conditions.

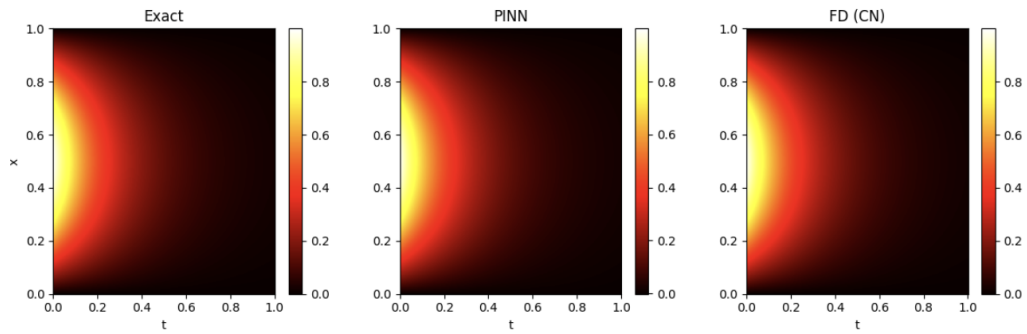


Figure 6.1: Heat map plots of the exact solution in comparison to the PINN and FD solutions.

To highlight where each method struggles, we examine heatmaps of the errors with respect to each solver. Figure 6.2 visualizes the absolute error $|u_{\theta}(x, t) - u_{\text{true}}(x, t)|$ for both methods.

Upon first glance, one might think that the finite-difference method is far less accurate, however, it is important to see that the scale in which the error is calculated for the FD method is several orders of magnitude lower than that of the PINN.

As shown in the figure, the PINN exhibits an almost wave-like error. By nature, the PINN is creating an approximation of the solution through the tanh activation

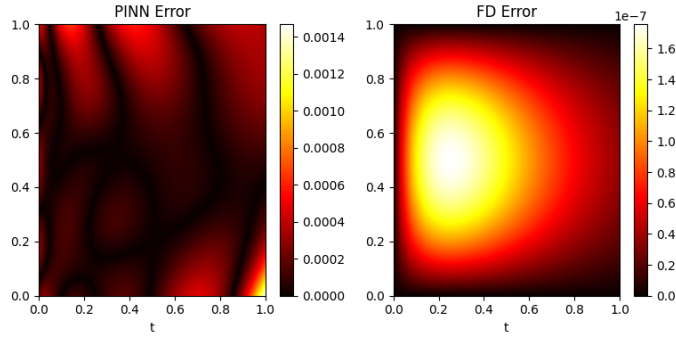


Figure 6.2: Absolute error relative to the analytical solution for the PINN and Crank–Nicolson (FD) methods over the spacetime domain.

functions. One plausible explanation for the oscillatory nature of the error is that these tanh functions have trouble fully approximating the exponential decay in the analytical solution.

In comparison, the FD method seemingly struggles the most with intermediate values of x and t . This is likely due to the fact that the finite-difference method builds the solution step by step starting from the initial and boundary values. Because of that, the farther a point is from those known values in the calculation process, the more chances there are for small errors to build up.

6.2 Quantitative Accuracy and Runtime Comparison

For a fair quantitative comparison of both methods, we consider accuracy and computational cost in the form of runtime. As described in Chapter 2, we evaluate accuracy using the relative L_2 error and the maximum error L_∞ . runtime is assessed

from the recorded training and inference times for the PINN and the total solve time for the CN finite-difference baseline. Results are summarized in table 6.2.

The finite-difference method is substantially more accurate than the PINN on this forward problem. The PINN achieves a relative L_2 error of 7.3322×10^{-4} and an L_∞ error of 1.4673×10^{-3} , whereas the Crank–Nicolson method attains a relative L_2 error of 3.3437×10^{-7} and an L_∞ error of 1.7558×10^{-7} . Clearly, the finite-difference baseline outperforms the PINN by several orders of magnitude in both overall and maximum error. This indicates that for this simple one-dimensional heat equation, the classical numerical solver provides a much closer approximation to the analytical solution.

Although the FD method clearly wins in pure accuracy, the runtime comparison shows a more interesting tradeoff. The total training time for the PINN is approximately 28.63 seconds, however, once trained, it evaluates relatively quickly with an inference time of 0.2162 seconds. In comparison, the FD method requires no training phase but takes 16.71 seconds to come to a solution. Although the PINN is slower overall if training is included, once trained, it is much faster than the FD method. In other words, from a latency standpoint, PINNs are a potentially more attractive choice for problems where repeated reuse of a trained model is important, whereas, for a one-time solution a finite-difference method is likely still the natural choice.

This comparison serves as a useful benchmark for illustrating the practical tradeoffs between PINNs and classical numerical solvers for problems like the one-dimensional heat equation. However, the results should not be interpreted as an absolute perfor-

Table 6.2: Compute cost and error comparison for the forward problem.

	PINN	Finite difference (CN)
Training time (s)	28.63	N/A
Inference time (s)	0.2162	16.7102
Rel. L^2 error	7.3322×10^{-4}	3.3437×10^{-7}
L^∞ error	1.4673×10^{-3}	1.7558×10^{-7}

mance limit for either of the two methods. In practice, additional refinement could likely improve accuracy for both approaches, whether through more advanced numerical techniques or through further optimization of the PINN architecture, hyperparameter tuning, and loss construction/weighting.

7 Inverse Problem

7.1 Inverse problem: formulation and setup

In the forward problem discussed in previous chapters, the thermal diffusivity parameter α is assumed to be the known value of $\alpha = 0.4$ with the goal of approximating the temperature $u(x, t)$. A natural extension of PINNs is the inverse problem, in which α is treated as unknown and must be inferred from a limited set of observations while still satisfying the governing PDE [1, 2].

As opposed to the forward problem where we have one equation to find one unknown u_θ given a specific x and t , if we are to attempt an approximation of values for two unknowns u_θ and α_θ we must incorporate some real data to come to a reliable result. As such, for the inverse problem, we assume access to a small dataset

$$D = \{(x_k, t_k, u_k)\}_{k=1}^{N_d}.$$

In this way, the neural network simultaneously learns a solution $u_\theta(x, t)$ that matches

the observed data and a parameter value α_θ that is consistent with the governing differential equation.

7.2 Loss function for the inverse problem

The inverse PINN uses the same heat-equation residual introduced earlier, with the only difference being that a known α is now replaced by the trainable parameter α_θ :

$$\text{res}_{\theta, \alpha_\theta}(x, t) = \frac{\partial u_\theta}{\partial t}(x, t) - \alpha_\theta \frac{\partial^2 u_\theta}{\partial x^2}(x, t).$$

As in the forward problem, the derivatives are computed using automatic differentiation [5].

To enforce a fit to the observed data an additional term is added to the loss function:

$$L_{\text{data}} = \frac{1}{N_d} \sum_{k=1}^{N_d} (u_\theta(x_k, t_k) - u_k)^2.$$

The full inverse objective is then written as

$$L = \lambda_1 L_{\text{PDE}} + \lambda_2 L_{\text{BC}} + \lambda_3 L_{\text{IC}} + \lambda_4 L_{\text{data}},$$

where L_{PDE} , L_{BC} , and L_{IC} are the same losses used in the forward problem. In principle, minimizing this new composite loss function allows the network to recover both a solution $u_\theta(x, t)$ and an estimate α_θ that is consistent with the available data and the underlying physics.

7.3 Discussion and future work

Although a full inverse PINN was not trained in the present project, the formulation above illustrates how naturally the framework extends beyond the forward problem. In contrast to classical numerical solvers, which typically assume known physical parameters and then solve the PDE, the PINN approach can incorporate sparse observations directly into the training objective and treat unknown coefficients as learnable quantities. This makes the inverse setting one of the most compelling potential applications of the PINN method [1, 2].

An inverse problem implementation would also make it possible to better study practical issues such as data quality, and the effect of loss weighting on parameter recovery. These questions are left for future work, but the inverse formulation presented here helps clarify why PINNs are attractive not only as forward PDE solvers, but also as tools for recovery of unknown physical constants.

8 Conclusion

This study examined the use of physics-informed neural networks for solving the one-dimensional heat equation and compared their performance with a classical finite-difference method. Both approaches successfully replicated the expected behavior of the system. However, the quantitative results showed that the finite-difference method achieved significantly higher accuracy, with errors several orders of magnitude smaller than that of the PINN. It also avoided the comparatively significant training cost required by the neural-network approach.

At the same time, this comparison provides a useful benchmark for illustrating the broader tradeoffs between PINNs and classical numerical solvers. While the classical method performed better on this simple forward problem, the PINN still produced a comparable approximation and displayed significantly faster inference after training was completed. Importantly, the results presented here reflect the specific implementations used in this study rather than the best achievable performance of either class of methods. Further refinement of either approach could likely improve accuracy.

Overall, these results suggest that classical numerical solvers still remain the more

effective choice for a straightforward problems such as the one-dimensional heat equation, where efficiency and accuracy are both critical. However, PINNs remain a promising framework due to their flexibility and their natural extension to inverse problems, parameter inference, and settings where physical constraints and sparse data must be incorporated simultaneously. Future work would likely focus on an inverse-problem implementation and further optimization of the PINN architecture and training procedure.

9 Bibliography

- [1] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs. *Journal of Computational Physics*, 378:686–707, 2019.
- [2] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
- [3] Zhuoyuan Liu, Yang Liu, Xin Yan, Wei Liu, Hang Nie, Shuaicheng Guo, and Chen Zhang. Automatic network structure discovery of physics informed neural networks via knowledge distillation. *Nature Communications*, 16:9558, 2025.
- [4] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022.

- [5] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
- [6] John Crank and Phyllis Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Mathematical Proceedings of the Cambridge Philosophical Society*, 43(1):50–67, 1947.
- [7] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics, Philadelphia, 2007.
- [8] Colby L. Wight and Jia Zhao. Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. *Communications in Computational Physics*, 29(3):930–954, 2021.
- [9] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.